VPN SSL/TLS y OpenVPN

Módulo 2: Seguridad en Acceso Remoto (VPN)

Máster en Seguridad TIC (XIII edición)

Equipo de Trabajo VPN SSL/TLS:

José Manuel Peña Ramírez

Juan Antonio Robledo Menéndez

Equipo de Trabajo OpenVPN:

Javier García Aguilar

José Manuel Peña Ramírez

Juan Antonio Robledo Menéndez

ÍNDICE

1. VPN Host-to-Host con SSL/TLS 3

1.1. Generación y análisis de una infraestructura VPN Host-to-Host para comunicación no segura 3

1.1.1. Host-to-host 3

1.1.2. Many-to-end 4

1.1.3. Análisis de tráfico de red en comunicaciones 6

1.2. Generación y análisis de una infraestructura VPN Host-to-Host para comunicación segura 7

1.3. Análisis de vulnerabilidades VPN 11

1.3.1. Ataques a los algoritmos criptográficos/ciphersuites usados 11

1.3.2. Ataques de Deny of Service (DoS) 14

2. Introducción a Virtual Private Networks con openVPN 17

2.1. Configuración del servidor openVPN 17

2.2. Configuración de un cliente openVPN 21

2.2.1. Linux 21

2.2.2. Windows 24

2.2.3. Android 27

2.3. OpenVPN Access Server 28

2.4. VPN personales (Tarea Extra) 28

1. VPN Host-to-Host con SSL/TLS

1.1. Generación y análisis de una infraestructura VPN Host-to-Host para comunicación no segura

1.1.1. Host-to-host

En este apartado, mediante código Java, se implementará un servidor de autenticación básico empleando sockets. Se mantendrá en escucha constante en el puerto 3343, y el programa esperará recibir una serie de credenciales para el establecimiento correcto de la conexión ('Angel' y 'qwerty', aunque se han modificado ambos por 'kali'). Por otro lado, tendremos un cliente que intentará abrir esa línea de comunicación antes mencionada mediante las credenciales.

El código de la práctica, tanto del 'ServerSocketVPN' como el de 'ClientSocketVPN', han sido modificados debido a las incompatibilidades existentes con las pequeñas ventanas emergentes (JOptionPane), en su defecto, se ha incluido un mecanismo de introducción de credenciales mediante consola.

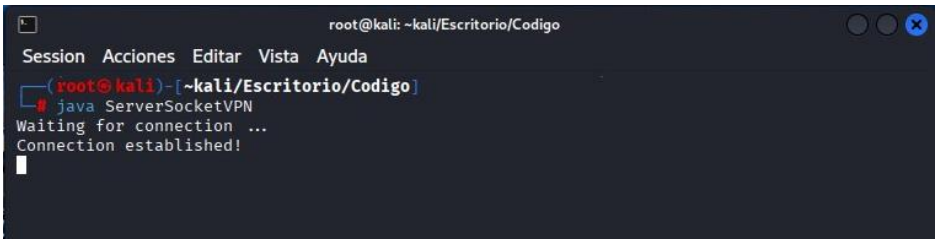


Figura 1. Conexión establecida con el cliente

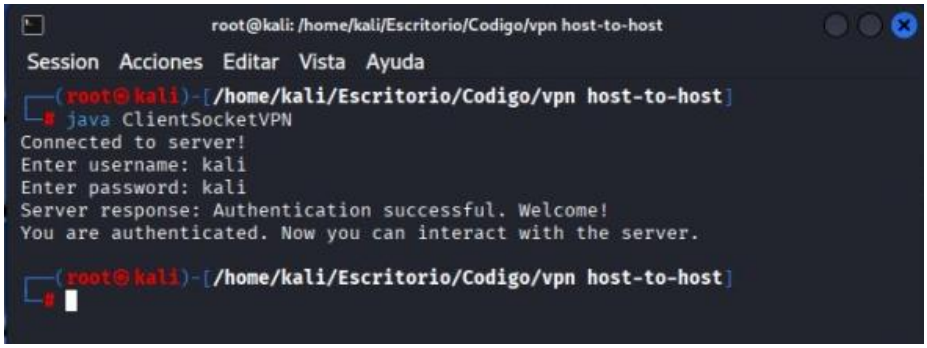


Figura 2. Conexión establecida con el servidor

Como se pueden apreciar en las capturas anteriores, la conexión ha sido exitosa, pudiendo observar un mensaje de bienvenida tras la introducción de las credenciales de forma correcta.

1.1.2. Many-to-end

En este apartado, el código se ha reescrito para soportar un modelo de procesamiento paralelo, solucionando la limitación más grave de la versión anterior. Mientras que antes el servidor solo podía atender a un cliente a la vez (bloqueando a todos los demás mientras uno estaba conectado), ahora la arquitectura permite gestionar múltiples usuarios de forma simultánea (véase la Figura 7).

Figura 3. Servidor Host-to-many a la espera de conexiones

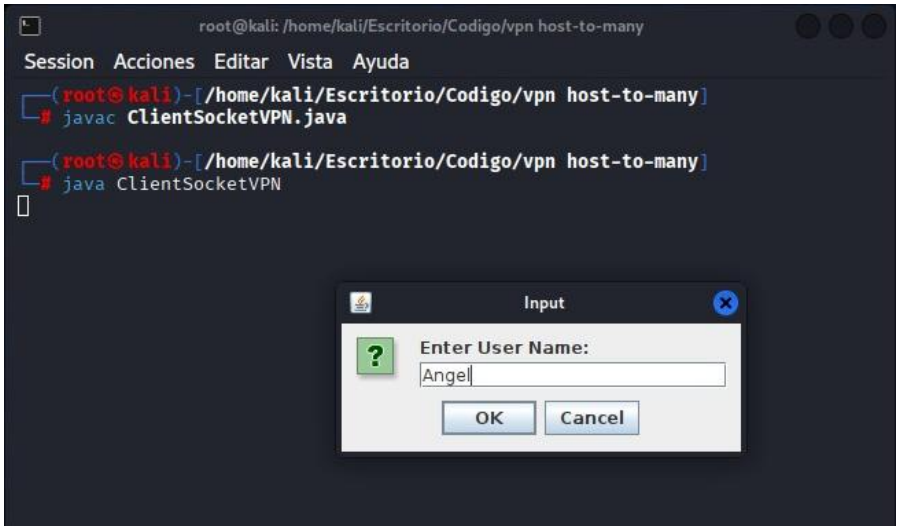


Figura 4. Conexión con el servidor mediante credenciales. Nombre de usuario

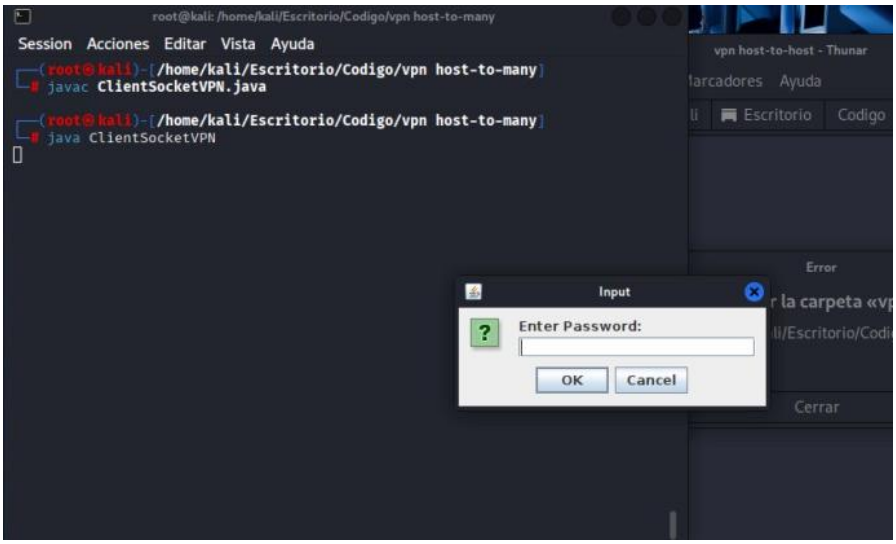


Figura 5. Conexión con el servidor mediante credenciales. Contraseña

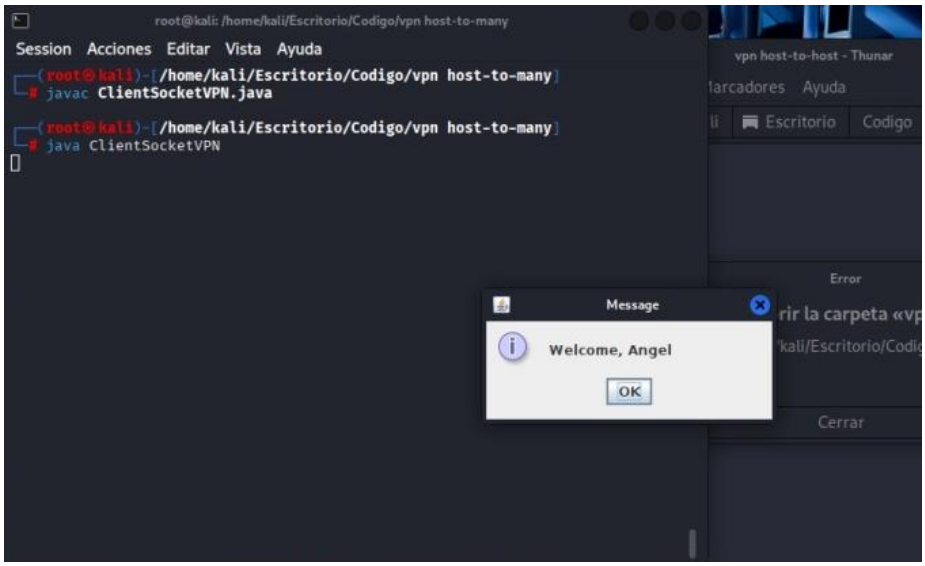


Figura 6. Conexión establecida con el servidor

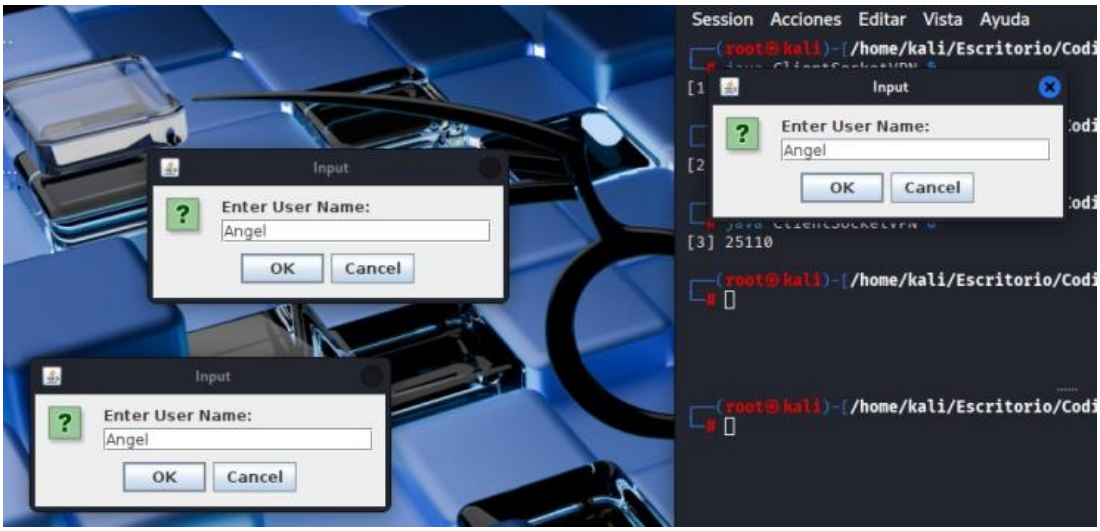


Figura 7. Solicitudes de credenciales en hilos concurrentes

1.1.3. Análisis de tráfico de red en comunicaciones

Otro detalle a destacar es que las credenciales pueden ser perfectamente obtenidas mediante el esnifado del tráfico (seguimiento del flujo TCP asociado al puerto 3343). Esto se debe a que el código sólo contempla la creación de sockets para la comunicación, pero debido a la ausencia de una capa de cifrado, no es posible una fase previa de intercambio de claves. Sin el handshake, no hay cifrado y por tanto los datos se enviarán a la red en texto plano.

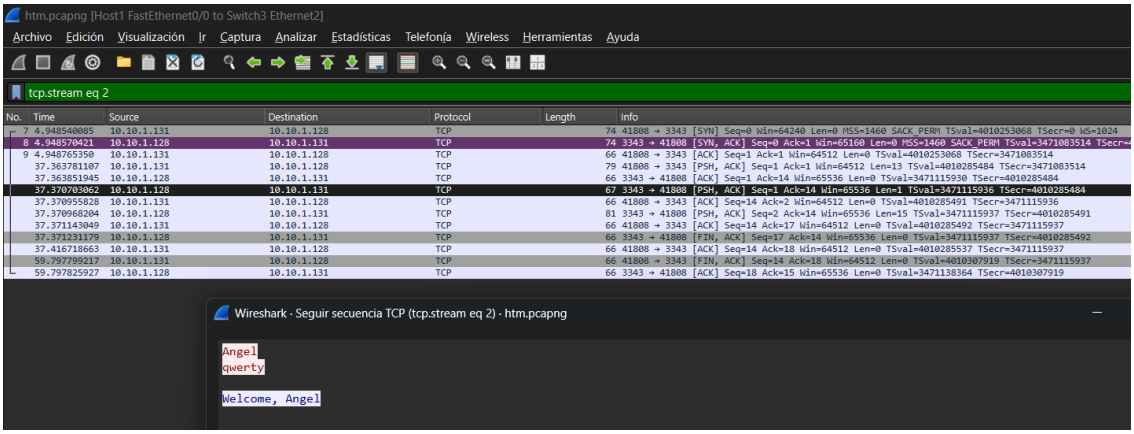


Figura 8. Seguimiento del flujo TCP asociado al puerto 3343

1.2. Generación y análisis de una infraestructura VPN Host-to-Host para comunicación segura

Para cumplir con la especificación del protocolo SSL/TLS, los servidores requieren certificados para su autenticación, lo cual en Java se gestiona mediante la creación de un almacén o repositorio de certificados de seguridad. Este proceso se simplifica con la herramienta nativa keytool, que permite generar dicho almacén.

Tras completar una serie de preguntas, se genera un archivo denominado keystore.jks que contiene una entrada de certificado bajo el alias "ssl", el cual servirá como el repositorio de confianza que utilizarán tanto los Sockets cliente como el servidor para establecer sus comunicaciones cifradadas.

```
┌──(root㉿kali)-[~kali/Escritorio/Codigo]
└─# keytool -genkey -keystore keystore.jks -alias ssl -keyalg RSA
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-compone
nt empty or press ENTER to use the default value in braces.
What is your first and last name?
  [Unknown]:  .
What is the name of your organizational unit?
  [Unknown]:  .
What is the name of your organization?
  [Unknown]:  .
What is the name of your City or Locality?
  [Unknown]:  .
What is the name of your State or Province?
  [Unknown]:  .
What is the two-letter country code for this unit?
  [Unknown]:  ES
Is C=ES correct?
  [no]:  YES

Generating 3.072 bit RSA key pair and self-signed certificate (SHA384withRSA)
 with a validity of 90 days
        for: C=ES
```

Figura 9. Creación de un almacén de certificados

La información que solicita keytool al crear el KeyStore (como el nombre, la unidad organizativa, la organización, la localidad y el código de país) tiene como objetivo principal establecer la identidad digital y garantizar la autenticidad del emisor del certificado.

```
┌──(root㉿kali)-[~kali/Escritorio/Codigo]
└─#  keytool -list -keystore keystore.jks
Enter keystore password:
Keystore type: PKCS12
Keystore provider: SUN

Your keystore contains 1 entry

ssl, 14 dic 2025, PrivateKeyEntry,
Certificate fingerprint (SHA-256): E8:BF:7D:B5:FB:AC:E6:36:FD:EC:81:4C:45:06:
BB:53:2E:C5:1A:27:87:C7:C8:E2:B4:4D:C2:63:E0:99:99:D9
```

Figura 10. Listado de las entradas del almacén de claves

A continuación, se generarán un servidor y un cliente VPN SSL, ambos ubicados en la misma máquina (pero con diferentes roles). Ambas harán uso de la keystore antes mencionada. Como se puede apreciar en capturas posteriores, la comunicación resulta ser exitosa.

```
Session  Acciones  Editar  Vista  Ayuda
┌──(root㉿kali)-[~kali/Escritorio/Codigo]
└─# java -Djavax.net.ssl.keyStore=./keystore.jks -Djavax.net.ssl.keyStorePass
word=123456 ServerSocketVPNSSL

Waiting for SSL connection ...
Server is waiting for connection ...
Connection established with client!
```

Figura 11. Conexión establecida con el cliente

```
┌──(root㉿kali)-[~kali/Escritorio/Codigo/vpn host-to-host]
└─# java -Djavax.net.ssl.trustStore=./keystore.jks -Djavax.net.ssl.trustStore
Password=123456 LoginClientVPNSSL
Connected to server!
Server response: Authentication successful. Welcome, admin!
```

Figura 12. Conexión establecida con el servidor

Es fundamental destacar que, en este caso concreto, sí se está utilizando el mismo archivo físico (keystore.jks), pero con roles semánticamente distintos. El servidor lo usa como KeyStore para presentar su identidad (su clave privada y certificado), mientras que el cliente lo usa como TrustStore para verificar que ese certificado es confiable.

Por otro lado, si se emplearan dos máquinas virtuales distintas, el archivo no podría ser el mismo físicamente. En el servidor se encontraría la KeyStore, la cual contendría la clave privada del servidor (que nunca debe ser compartida) y su certificado público; y en el cliente, estaría la TrustStore; en este caso, sólo necesitaría contener el certificado público del servidor (o de la entidad que lo firmó) para poder validar la conexión.

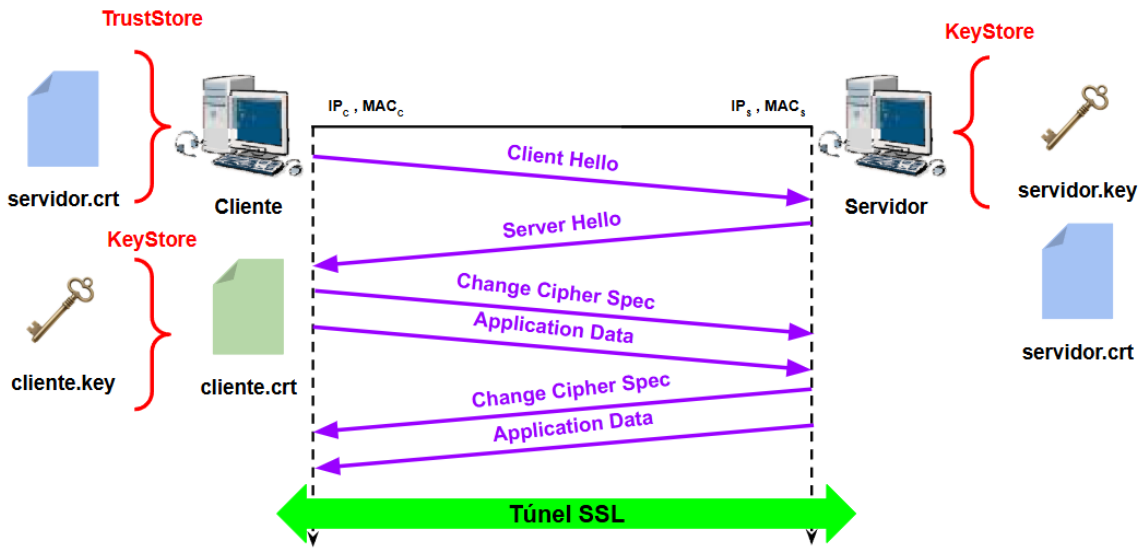


Figura 13. Esquema de distribución de KeyStores y certificados y diagrama de paso de mensajes

Si realizamos de nuevo un seguimiento del flujo del tráfico TCP, observamos como las credenciales ya no se intercambian mediante texto plano, en este caso van cifradas bajo el algoritmo seleccionado por el servidor (SERVER_HELLO como se analizará más adelante). Ya aparecen cabeceras TLSv1.3.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.10.1.131	10.10.1.128	TCP	74	50294 → 7070 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1243262389 TSecr=0 WS=1024
2	0.000033250	10.10.1.128	10.10.1.131	TCP	74	7070 → 50294 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=807803918 TSecr=1243262389 WS=1024
3	0.000220040	10.10.1.131	10.10.1.128	TCP	66	50294 → 7070 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=1243262389 TSecr=807803918
4	0.030371445	10.10.1.131	10.10.1.128	TLSv1.3	506	Client Hello
5	0.030400029	10.10.1.128	10.10.1.131	TCP	66	7070 → 50294 [ACK] Seq=1 Ack=441 Win=65536 Len=0 TSval=807803948 TSecr=1243262419
6	0.038592844	10.10.1.128	10.10.1.131	TLSv1.3	193	Server Hello
7	0.038852259	10.10.1.131	10.10.1.128	TCP	66	50294 → 7070 [ACK] Seq=441 Ack=128 Win=64512 Len=0 TSval=1243262428 TSecr=807803956
8	0.041245658	10.10.1.128	10.10.1.131	TLSv1.3	72	Change Cipher Spec
9	0.041456865	10.10.1.131	10.10.1.128	TCP	66	50294 → 7070 [ACK] Seq=441 Ack=134 Win=64512 Len=0 TSval=1243262430 TSecr=807803959
	0.042267609	10.10.1.128	10.10.1.131	TLSv1.3	136	Application Data
	0.042418024	10.10.1.131	10.10.1.128	TCP	66	50294 → 7070 [ACK] Seq=441 Ack=204 Win=64512 Len=0 TSval=1243262431 TSecr=807803960
	0.042630231	10.10.1.131	10.10.1.128	TLSv1.3	72	Change Cipher Spec
	0.043825473	10.10.1.128	10.10.1.131	TLSv1.3	1079	Application Data
	0.088967102	10.10.1.131	10.10.1.128	TCP	66	50294 → 7070 [ACK] Seq=447 Ack=1217 Win=64512 Len=0 TSval=1243262478 TSecr=807803962
	0.088995227	10.10.1.128	10.10.1.131	TLSv1.3	586	Application Data, Application Data
	0.089174476	10.10.1.131	10.10.1.128	TCP	66	50294 → 7070 [ACK] Seq=447 Ack=1737 Win=64512 Len=0 TSval=1243262478 TSecr=807804007
	0.092407077	10.10.1.131	10.10.1.128	TLSv1.3	156	Application Data
	0.095308098	10.10.1.128	10.10.1.131	TLSv1.3	1345	Application Data
	0.095502472	10.10.1.131	10.10.1.128	TLSv1.3	157	Application Data, Application Data
	0.097744830	10.10.1.128	10.10.1.131	TLSv1.3	147	Application Data
	0.098171869	10.10.1.128	10.10.1.131	TLSv1.3	146	Application Data, Application Data
	0.098851031	10.10.1.131	10.10.1.128	TLSv1.3	106	Application Data
	0.098874697	10.10.1.128	10.10.1.131	TCP	54	7070 → 50294 [RST] Seq=3178 Win=0 Len=0
	0.098935488	10.10.1.131	10.10.1.128	TLSv1.3	106	Application Data
	0.098940655	10.10.1.128	10.10.1.131	TCP	54	7070 → 50294 [RST] Seq=3178 Win=0 Len=0

Figura 14. Captura del tráfico intercambiado

Figura 15. Seguimiento del flujo TCP asociado al puerto 7070

A continuación, se han empleado las herramientas de exportación e importación de certificados para comunicar dos máquinas remotas, obteniendo, de nuevo, una comunicación exitosa.

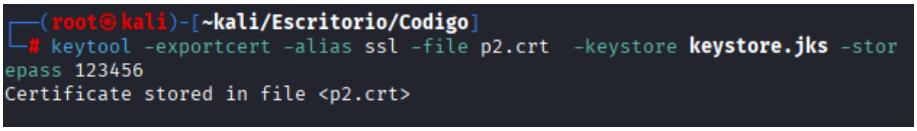


Figura 16. Exportación del certificado

```
┌──(root㉿kali)-[~kali/Escritorio/Codigo/vpn host-to-host]
└─# keytool -importcert -alias ssl -file p2.crt  -keystore keystore.jks -stor
epass 123456
Owner: C=ES
Issuer: C=ES
Serial number: f16951dc3e7636a9
Valid from: Sun Dec 14 01:03:02 CET 2025 until: Sat Mar 14 01:03:02 CET 2026
Certificate fingerprints:
         SHA1: 2B:B5:BE:46:99:ED:0B:D3:FB:61:DC:D3:14:23:7C:75:71:8B:8A:B5
         SHA256: E8:BF:7D:B5:FB:AC:E6:36:FD:EC:81:4C:45:06:BB:53:2E:C5:1A:27:
87:C7:C8:E2:B4:4D:C2:63:E0:99:99:D9
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 3072-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 78 07 DB A4 E7 60 16 BB   75 8D 48 08 96 BD 46 0F  x....`..u.H...F.
0010: E0 C4 FB F5                                        ....
]
]

Trust this certificate? [no]:  yes
Certificate was added to keystore
```

Figura 17. Importación del certificado

1.3. Análisis de vulnerabilidades VPN

1.3.1. Ataques a los algoritmos criptográficos/ciphersuites usados

Basándonos en el tráfico previamente generado, al analizar el mensaje Client Hello se pueden observar las distintas cipher suites que soporta el equipo cliente, todas ellas compatibles con TLS 1.2, tal y como se aprecia en la cabecera. Algunas de estas suites pueden considerarse seguras (ya que continúan siendo válidas o tienen equivalentes en TLS 1.3), mientras que otras han quedado obsoletas por el uso de algoritmos criptográficos menos robustos.

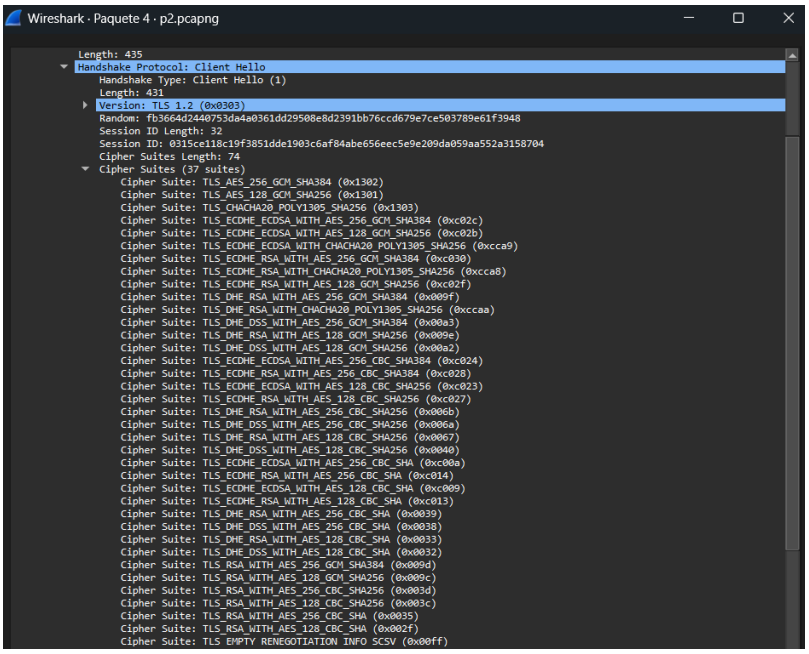


Figura 18. Ciphersuites soportados por el cliente

En nuestro escenario, si analizamos las cabeceras del mensaje Server Hello, que incluye la cipher suite seleccionada por el servidor para establecer la comunicación entre el cliente y el servidor VPN, se observa que la suite elegida es TLS_AES_256_GCM_SHA384. Esta cipher suite es considerada segura porque utiliza cifrado simétrico AES con clave de 256 bits en modo GCM, que proporciona confidencialidad e integridad de los datos, junto con la función hash SHA-384, que ofrece una alta resistencia criptográfica. Además, al ser una suite propia de TLS 1.3, elimina algoritmos inseguros y simplifica el proceso de negociación, reduciendo la superficie de ataque y mejorando la seguridad general de la comunicación.

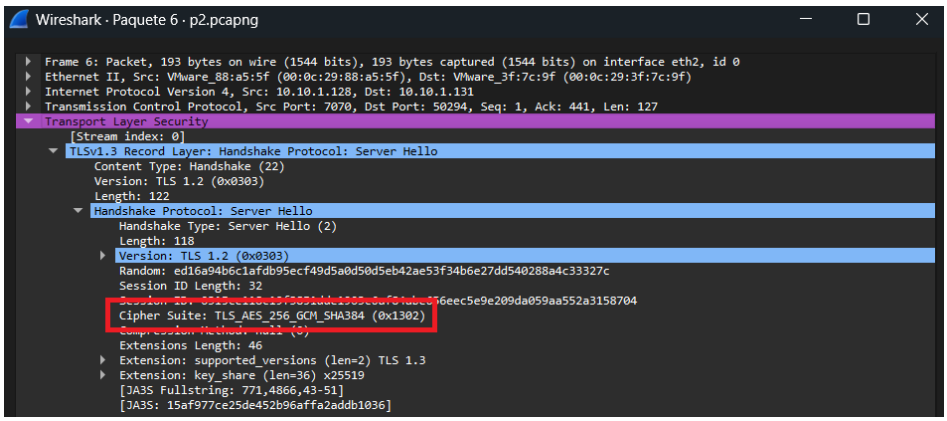


Figura 19. Ciphersuite seleccionado por el servidor

En la memoria se propone la aplicación de dos cipher suites diferentes: una considerada segura y otra insegura. En nuestro caso, el cifrado seguro podría ser perfectamente el ya mencionado anteriormente (TLS_AES_256_GCM_SHA384). Sin embargo, para la puesta en marcha del cifrado inseguro se ha optado por utilizar TLS_RSA_WITH_AES_128_CBC_SHA, ya que emplea un intercambio de claves basado en RSA sin forward secrecy y un modo de cifrado CBC, el cual es más vulnerable a distintos tipos de ataques criptográficos conocidos (como padding oracle attacks). Además, utiliza SHA-1 como función hash, algoritmo que actualmente se considera obsoleto debido a debilidades criptográficas, lo que hace que esta cipher suite no cumpla con los estándares de seguridad modernos y sea desaconsejada en entornos actuales.

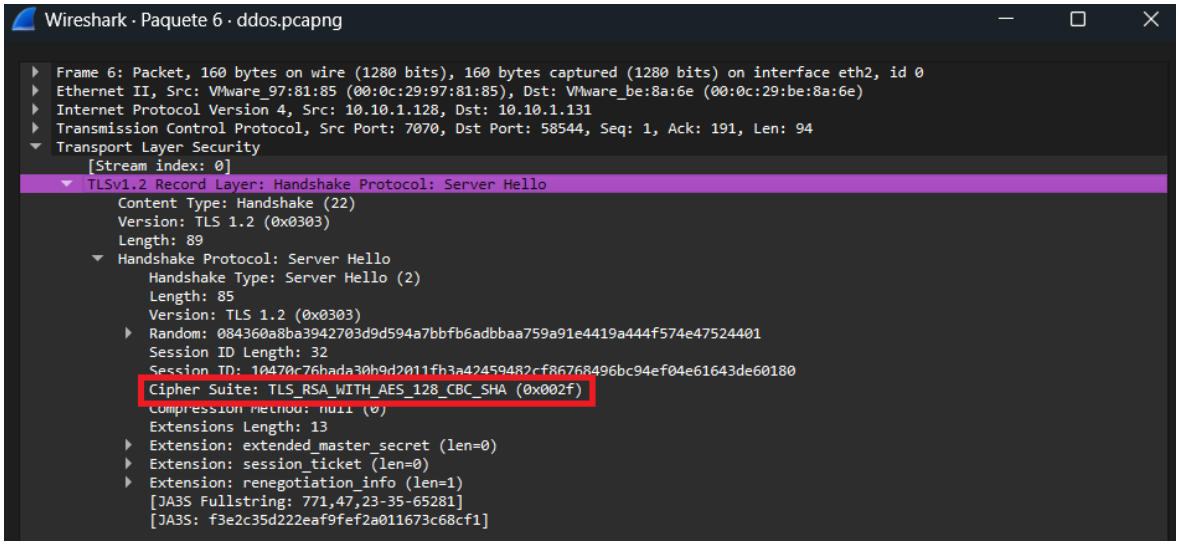


Figura 20. Ciphersuite inseguro seleccionado por el servidor

A continuación, se muestra el código empleado para forzar el uso de determinadas ciphersuites (seguras/inseguras) tanto en el lado del cliente como del servidor.

```
SSLSocketFactory factory = (SSLSocketFactory) SSLSocketFactory.getDefault();
SSLSocket socket = (SSLSocket) factory.createSocket("10.10.1.128", 3343);
socket.setEnabledProtocols(new String[]{"TLSv1.2"});
socket.setEnabledCipherSuites(new String[]{
    "TLS_RSA_WITH_AES_128_CBC_SHA"
});
```

Tabla 1. Código modificado en el cliente para forzar el uso de una ciphersuite insegura

```
serverSocket.setEnabledProtocols(new String[]{
    "TLSv1.3",
    "TLSv1.2"
});
serverSocket.setEnabledCipherSuites(new String[]{
    // TLS 1.3
    "TLS_AES_256_GCM_SHA384",
    "TLS_AES_128_GCM_SHA256",
    // TLS 1.2
    "TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384",
    "TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256"
});
```

Tabla 2. Código modificado en el servidor para forzar el uso de ciphersuites seguras

1.3.2. Ataques de Deny of Service (DoS)

Nuestro entorno de pruebas para el ataque DoS parte del escenario descrito en el apartado anterior, compuesto por un servidor VPN y un cliente VPN, al que se añade un tercer equipo dentro de la red que asumirá el rol de atacante. Este último llevará a cabo un ataque de denegación de servicio mediante un TCP SYN Flood, utilizando la herramienta hping3, con el objetivo de impedir que el cliente pueda establecer conexión con el servidor VPN.

Los resultados obtenidos reflejan la fragilidad de nuestro código a la hora de proteger la conexión frente a este tipo de ataques, ya que, como se puede observar, el cliente no consigue conectarse al servidor debido a que este último se encuentra ocupado procesando las múltiples peticiones TCP enviadas por el atacante. Más adelante se describirán los distintos mecanismos empleados para la mitigación de este tipo de ataques.

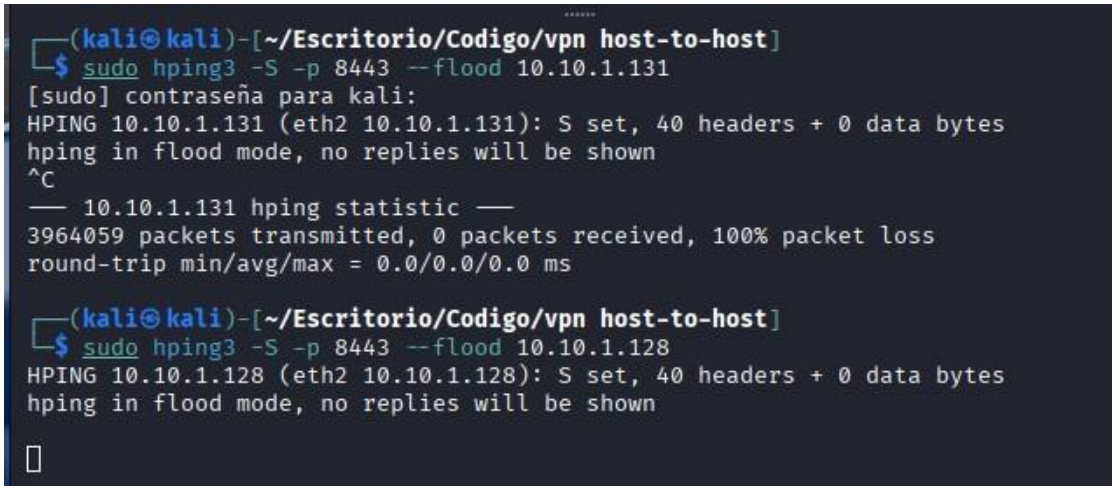


Figura 21. Ataque de denegación de servicio (DoS) al servidor VPN

```
┌──(root㉿kali)-[~kali/Escritorio/Codigo]
└─# java -Djavax.net.ssl.keyStore=./keystore.jks -Djavax.net.ssl.keyStorePassword=123456 ServerSocketVPNSSL

Waiting for SSL connection ...
Server is waiting for connection ...
```

Figura 22. Indisponibilidad del servidor debido al ataque de denegación de servicio (DoS)

```
┌──(root㉿kali)-[~kali/Escritorio/p2]
└─# java -Djavax.net.ssl.trustStore=./keystore.jks -Djavax.net.ssl.trustStore
Password=123456 LoginClientVPNSSL
```

Figura 23. Imposibilidad del cliente de establecer conexión debido al ataque de denegación de servicio (DoS)

Existen diferentes mecanismos para mitigar ataques de denegación de servicio (DoS), los cuales pueden clasificarse principalmente en soluciones a nivel de software y en ajustes a nivel de kernel del sistema operativo, en este caso Linux, donde se aloja el servidor VPN.

A nivel de aplicación, se ha implementado un control de conexiones concurrentes mediante el uso de un pool de hilos gestionado por ExecutorService, limitando el número máximo de hilos activos para evitar el agotamiento de recursos del servidor. Adicionalmente, se incorpora un mecanismo de rate limiting por dirección IP, donde se registra el número de conexiones realizadas por cada cliente dentro de una ventana temporal definida. Si una IP supera el umbral máximo de conexiones permitidas (MAX_CONN_PER_IP) en dicho intervalo (WINDOW_MS), la conexión es rechazada de forma inmediata. Este enfoque permite identificar comportamientos anómalos propios de ataques DoS y bloquearlos antes de que consuman recursos críticos del servidor, reduciendo así el impacto del ataque sobre clientes legítimos. El establecimiento de un timeout en el socket (setSoTimeout) contribuye además a liberar conexiones inactivas o maliciosas que no completan correctamente el handshake TLS.

```
ExecutorService pool = Executors.newFixedThreadPool(MAX_THREADS);
while (true) {
    SSLSocket client = (SSLSocket) serverSocket.accept();
    String clientIP = client.getInetAddress().getHostAddress();
    long now = System.currentTimeMillis();
    ConnectionInfo info = ipMap.computeIfAbsent(clientIP, k -> new ConnectionInfo());
    synchronized (info) {
        if (now - info.windowStart > WINDOW_MS) {
            info.count.set(0);
            info.windowStart = now;
        }
        if (info.count.incrementAndGet() > MAX_CONN_PER_IP) {
            System.out.println("Conexión rechazada por DoS desde IP: " + clientIP);
            client.close();
            continue;
        }
    }
    client.setSoTimeout(5000);
    pool.execute(new ClientHandler(client));
}
```

Tabla 3. Código modificado en el servidor de control de conexiones para mitigar DoS

Complementariamente, a nivel de sistema operativo, se ha activado el mecanismo de TCP SYN Cookies en el kernel de Linux. Esta técnica está diseñada específicamente para mitigar ataques de tipo TCP SYN Flood, en los que el atacante envía un gran número de paquetes SYN sin completar el three-way handshake, saturando la cola de conexiones semiabiertas del servidor. Al habilitar SYN Cookies, el kernel evita asignar recursos a estas conexiones incompletas, codificando la información del estado de la conexión directamente en el número de secuencia del paquete SYN-ACK. De este modo, solo cuando el cliente responde correctamente con un ACK válido, el servidor reserva memoria y establece la conexión. Esta medida reduce significativamente el consumo de recursos ante ataques volumétricos y, combinada con los controles a nivel de aplicación, proporciona una defensa en profundidad que incrementa la resiliencia del servidor VPN frente a ataques DoS.

```
┌──(root㉿kali)-[~]
└─# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1

┌──(root㉿kali)-[~]
└─# sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 1
```

Figura 24. Activación del TCP SYN Cookies en el kernel

2. Introducción a Virtual Private Networks con openVPN

En esta segunda parte de la práctica se emplea OpenVPN, basado en OpenSSL, para crear túneles VPN seguros que encapsulan el tráfico IP sobre UDP o TCP mediante un modelo cliente-servidor. La autenticación se realizará mediante certificados digitales y un token secreto compartido para el intercambio de claves. El entorno incluye clientes Linux y Windows que se conectan de forma segura a una red corporativa a través de un servidor OpenVPN.

2.1. Configuración del servidor openVPN

En primer lugar, desde el sistema que actuará como servidor, se procede a la creación de los certificados digitales utilizando los scripts easy-rsa incluidos en el paquete OpenVPN. Tras copiar y preparar el entorno de trabajo, se edita el fichero vars, que define los parámetros globales utilizados durante la generación de los certificados. En este paso es especialmente importante modificar las variables del archivo vars, adaptando campos como país, organización o nombre común a los datos del alumno, ya que estos valores quedarán incrustados en los certificados y serán utilizados para la autenticación dentro del túnel VPN.

```
root@kali: /etc/openvpn/easy-rsa
Session  Actions  Edit  View  Help
  GNU nano 8.7                                                    vars *

# Define X509 DN mode.
#
# This is used to adjust which elements are included in the Subject field
# as the DN ("Distinguished Name"). Note that in 'cn_only' mode the
# Organizational fields, listed further below, are not used.
#
# Choices are:
#   cn_only  - Use just a commonName value.
#   org      - Use the "traditional" format:
#              Country/Province/City/Org/Org.Unit/email/commonName
#

set_var EASYRSA_DN      "org"

# Organizational fields (used with "org" mode and ignored in "cn_only" mode).
# These are the default values for fields which will be placed in the
# certificate.  Do not leave any of these fields blank, although interactively
# you may omit any specific field by typing the "." symbol (not valid for
# email).
#
# NOTE: The following characters are not supported
#       in these "Organizational fields" by Easy-RSA:
#       back-tick (`)
#
set_var EASYRSA_REQ_COUNTRY     "ES"
set_var EASYRSA_REQ_PROVINCE    "Andalucia"
set_var EASYRSA_REQ_CITY        "Sevilla"
set_var EASYRSA_REQ_ORG "Universidad de Sevilla"
set_var EASYRSA_REQ_EMAIL       "juarobmen@alum.us.es"
set_var EASYRSA_REQ_OU          "VPN"
```

Figura 25. Configuración de las variables de los certificados digitales

Se configura la comunicación segura mediante SSL/TLS creando la infraestructura de clave pública (PKI) con EasyRSA, generando parámetros Diffie-Hellman, el certificado de la autoridad (CA) y los certificados del servidor. Se firma el certificado del servidor y se genera una clave secreta adicional (ta.key) para mitigar ataques DDoS. Al finalizar, el directorio /etc/openvpn/easy-rsa contiene todas las claves y certificados necesarios para el túnel VPN.

```
root@kali: /etc/openvpn/easy-rsa/pki/private
Session  Actions  Edit  View  Help

┌──(root㉿kali)-[/etc/openvpn/easy-rsa/pki]
└─# ls -l
total 56
-rw------- 1 root root 1744 Jan 29 13:39 ca.crt
drwx------ 2 root root 4096 Jan 29 13:40 certs_by_serial
-rw------- 1 root root  428 Jan 29 13:38 dh.pem
-rw------- 1 root root  167 Jan 29 13:40 index.txt
-rw------- 1 root root   20 Jan 29 13:40 index.txt.attr
-rw------- 1 root root    0 Jan 29 13:38 index.txt.old
drwx------ 3 root root 4096 Jan 29 13:40 inline
drwx------ 2 root root 4096 Jan 29 13:40 issued
drwx------ 2 root root 4096 Jan 29 13:39 private
drwx------ 2 root root 4096 Jan 29 13:39 reqs
drwx------ 4 root root 4096 Jan 29 13:38 revoked
-rw------- 1 root root   33 Jan 29 13:40 serial
-rw------- 1 root root   33 Jan 29 13:40 serial.old
-rw------- 1 root root 5821 Jan 29 13:36 vars.example
```

```
┌──(root㉿kali)-[/etc/openvpn/easy-rsa/pki/issued]
└─# ls -l
total 8
-rw------- 1 root root 5350 Jan 29 13:40 miservidor.crt
```

```
┌──(root㉿kali)-[/etc/openvpn/easy-rsa/pki/private]
└─# ls -l
total 8
-rw------- 1 root root 1886 Jan 29 13:38 ca.key
-rw------- 1 root root 1704 Jan 29 13:39 miservidor.key
```

Figura 26. Generación de las claves y certificados

Una vez lanzada la configuración del servidor, podemos iniciar OpenVPN con 'openvpn --config /etc/openvpn/server.conf'. El servidor comenzará a escuchar en el puerto UDP 1194, tal como se especifica en la configuración, y para verificar que se ha iniciado correctamente podemos utilizar herramientas como netstat o nmap.

Por ejemplo, ejecutando 'nmap -p 1194 localhost' se obtiene información sobre el puerto y el estado del servicio, confirmando que OpenVPN está activo y listo para aceptar conexiones entrantes, lo que permite asegurarnos de que la configuración del servidor se ha cargado correctamente antes de conectar los clientes.

```
┌──(kali㉿kali)-[~]
└─$ nmap -p1194 localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-29 13:53 -0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000082s latency).
Other addresses for localhost (not scanned): ::1

PORT     STATE  SERVICE
1194/tcp closed openvpn

Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds
```

Figura 27. Puerto 1194 asociado al servicio 'openvpn'

Dentro de nuestro servidor OpenVPN es posible desplegar servicios corporativos, como un portal interno de la empresa. En nuestro caso, se procederá a instalar y activar un servidor Apache para ofrecer este tipo de servicios.

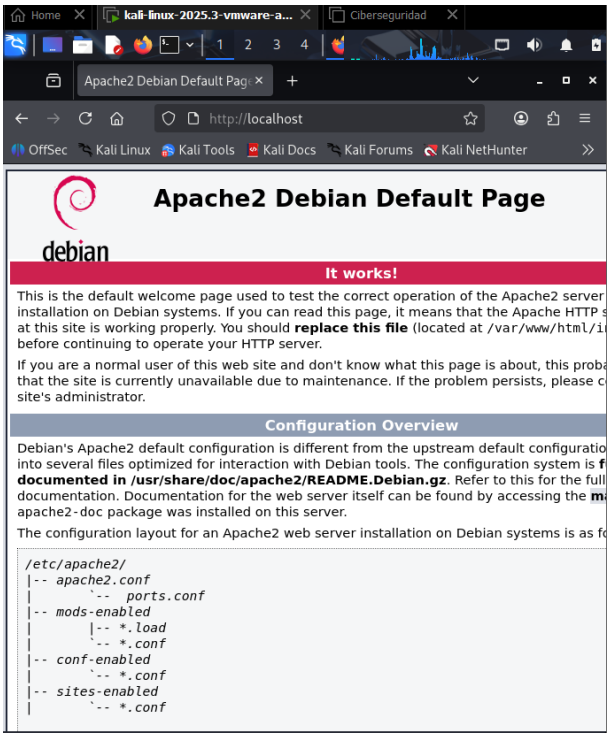


Figura 28. Comprobación del funcionamiento del servidor Apache (desde el servidor VPN)

Si se listan las interfaces del equipo servidor, se puede observar la presencia de una interfaz tun0, la cual actúa como el punto de terminación del túnel virtual. Utiliza el driver Universal TUN/TAP, permitiendo que los paquetes de la capa de red (IP) sean procesados por el espacio de usuario (OpenVPN) para su posterior cifrado y transporte mediante UDP/1194.

```
┌──(root㉿kali)-[~]
└─# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:91:17:7d brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.148/24 brd 192.168.1.255 scope global dynamic noprefixrout
e eth0
       valid_lft 84688sec preferred_lft 84688sec
    inet6 fe80::6af1:299d:6e5:2db3/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN
 group default qlen 1000
    link/none
    inet 10.8.0.1/24 brd 10.8.0.255 scope global tun0
       valid_lft forever preferred_lft forever
    inet6 fe80::8750:afdb:8d5e:d710/64 scope link stable-privacy proto kernel
_ll
       valid_lft forever preferred_lft forever
```

Figura 29. Creación de una interfaz virtual de túnel

Existen diversos parámetros de configuración de openVPN en el lado del servidor que permiten modificar las características de las conexiones con los clientes y/o red corporativa además de hacer la conexión más segura:

- Topología y enrutamiento de la VPN
  - **push route X.X.X.X Y.Y.Y.Y**: Permite insertar rutas en los clientes. Por ejemplo, push "route 192.168.1.0 255.255.255.0" hace que el cliente enrute el tráfico a la red corporativa.
  - **client-to-client**: Habilita la comunicación directa entre clientes conectados (útil si varios clientes deben verse entre sí).
- Autenticación TLS y certificados
  - **ncp-ciphers**: permite negociar con el cliente el mejor algoritmo soportado.
- Opciones de seguridad avanzadas
  - **user nobody** y **group nogroup**: Después de iniciar, openVPN puede descender privilegios al usuario/grupo sin permisos especiales. Así, en caso de explotación, el proceso compromete menos recursos críticos
  - **reneg-sec X**: Fuerza renegociación de claves cada X segundos, renovando las claves periódicamente. Esto mejora la seguridad al limitar cuánto tiempo usa la misma clave simétrica.

```
port 1194
proto udp
dev tun
ca /etc/openvpn/keys/ca.crt
cert /etc/openvpn/keys/miservidor.crt
```

```
key /etc/openvpn/keys/miservidor.key
dh /etc/openvpn/keys/dh.pem
server 10.8.0.0 255.255.255.0
ifconfig-pool-persist /var/log/openvpn/ipp.txt
keepalive 10 120
topology subnet
tls-auth /etc/openvpn/keys/ta.key 0
cipher AES-256-GCM
persist-key
persist-tun
status /var/log/openvpn/openvpn-status.log
verb 9 # 0-9, 9 más verboso
auth SHA512
tls-version-min 1.2
tls-cipher TLS-DHE-RSA-WITH-AES-256-GCM-SHA384
push "route 192.168.1.0 255.255.255.0"
client-to-client #opcional
user nobody
group nobody
reneg-sec 3600
```

Tabla 4. Modificación de la configuración del servidor

Si bien se podría forzar el uso de la versión TLS 1.3, la realidad es que en general el servidor cuenta con un gran número de parámetros que hacen que la conexión sea completamente segura.

2.2. Configuración de un cliente openVPN

2.2.1. Linux

En el equipo cliente se han importado de forma segura los archivos necesarios para la conexión VPN, incluyendo micliente1.crt, micliente1.key, ta.key y ca.crt, los cuales se han colocado en la carpeta /etc/openvpn/keys/ para que puedan ser utilizados por la configuración de OpenVPN.

```
┌──(root㉿kali)-[/etc/openvpn/keys]
└─# ls -l
total 20
-rw------- 1 root root 1744 Jan 29 14:44 ca.crt
-rw------- 1 root root 5332 Jan 29 14:44 micliente1.crt
-rw------- 1 root root 1704 Jan 29 14:44 micliente1.key
-rw------- 1 root root  636 Jan 29 14:44 ta.key
```

Figura 30. Ficheros importados en el equipo cliente

Tras lanzar el cliente con el comando ‘openvpn --config /etc/openvpn/client.conf’, se puede apreciar como en el negociado de la conexión se ha llegado a una configuración de canal

cifrado mediante un CipherSuite TLS_AES_256_GCM_SHA384 y un protocolo TLSv1.3 con claves RSA de 2048 bits. La IP de la conexión par es la 192.168.1.148.

```
┌──(root㉿kali)-[/etc/openvpn]
└─# openvpn --config /etc/openvpn/client.conf
2026-01-29 14:50:03 Note: Kernel support for ovpn-dco missing, disabling data chan
nel offload.
2026-01-29 14:50:03 OpenVPN 2.6.14 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4]
 [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD] [DCO]
2026-01-29 14:50:03 library versions: OpenSSL 3.5.4 30 Sep 2025, LZO 2.10
2026-01-29 14:50:03 DCO version: N/A
2026-01-29 14:50:03 TCP/UDP: Preserving recently used remote address: [AF_INET]192
.168.1.148:1194
2026-01-29 14:50:03 Socket Buffers: R=[212992→212992] S=[212992→212992]
2026-01-29 14:50:03 UDPv4 link local: (not bound)
2026-01-29 14:50:03 UDPv4 link remote: [AF_INET]192.168.1.148:1194
2026-01-29 14:50:03 TLS: Initial packet from [AF_INET]192.168.1.148:1194, sid=64bd
47c8 4cc82699
2026-01-29 14:50:03 VERIFY OK: depth=1, C=ES, ST=Andalucia, L=Sevilla, O=Universid
ad de Sevilla, OU=VPN, CN=VPN CA, emailAddress=juarobmen@alum.us.es
2026-01-29 14:50:03 VERIFY KU OK
2026-01-29 14:50:03 Validating certificate extended key usage
2026-01-29 14:50:03 ++ Certificate has EKU (str) TLS Web Server Authentication, ex
pects TLS Web Server Authentication
2026-01-29 14:50:03 VERIFY EKU OK
2026-01-29 14:50:03 VERIFY OK: depth=0, C=ES, ST=Andalucia, L=Sevilla, O=Universid
ad de Sevilla, OU=VPN, CN=miservidor, emailAddress=juarobmen@alum.us.es
2026-01-29 14:50:03 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA38
4, peer certificate: 2048 bits RSA, signature: RSA-SHA256
2026-01-29 14:50:03 [miservidor] Peer Connection Initiated with [AF_INET]192.168.1
.148:1194
2026-01-29 14:50:03 TLS: move_session: dest=TM_ACTIVE src=TM_INITIAL reinit_src=1
2026-01-29 14:50:03 TLS: tls_multi_process: initial untrusted session promoted to
trusted
2026-01-29 14:50:03 PUSH: Received control message: 'PUSH_REPLY,route-gateway 10.8
.0.1,topology subnet,ping 10,ping-restart 120,ifconfig 10.8.0.2 255.255.255.0,peer
-id 0,cipher AES-256-GCM,protocol-flags cc-exit tls-ekm dyn-tls-crypt,tun-mtu 1500
'
2026-01-29 14:50:03 OPTIONS IMPORT: --ifconfig/up options modified
2026-01-29 14:50:03 OPTIONS IMPORT: route-related options modified
2026-01-29 14:50:03 OPTIONS IMPORT: tun-mtu set to 1500
2026-01-29 14:50:03 TUN/TAP device tun0 opened
2026-01-29 14:50:03 net_iface_mtu_set: mtu 1500 for tun0
2026-01-29 14:50:03 net_iface_up: set tun0 up
2026-01-29 14:50:03 net_addr_v4_add: 10.8.0.2/24 dev tun0
2026-01-29 14:50:03 Initialization Sequence Completed
2026-01-29 14:50:03 Data Channel: cipher 'AES-256-GCM', peer-id: 0
2026-01-29 14:50:03 Timers: ping 10, ping-restart 120
2026-01-29 14:50:03 Protocol options: protocol-flags cc-exit tls-ekm dyn-tls-crypt
```

Figura 31. Conexión con el servidor VPN exitosa

Figura 32. Acceso a los servicios desplegados en la red corporativa (desde el cliente)

La conexión establecida en el escenario de pruebas es altamente segura y robusta, cumpliendo con los estándares de cifrado de grado militar actuales. Al emplear TLSv1.3, se eliminan protocolos obsoletos y vulnerabilidades heredadas, mientras que el uso de AES-256-GCM garantiza no solo una confidencialidad impenetrable para la computación convencional, sino también la integridad de los datos en tiempo real gracias a su modo de autenticación galois. El uso de SHA384 y claves RSA de 2048 bits (el estándar recomendado por el NIST) asegura una autenticación de dispositivos sólida y una protección contra ataques de suplantación, logrando un túnel VPN que equilibra perfectamente un rendimiento optimizado con una capacidad de proteger el tráfico contra interceptaciones en redes públicas.

Existen diversos parámetros de configuración de openVPN en el lado del cliente que permiten modificar las características de las conexiones con el servidor además de hacer la conexión más segura:

- Renegociación de claves
    - **reneg-bytes n**: renegocia tras transmitir n bytes (deshabilitado por defecto salvo claves <128 bits, donde el valor por defecto es 64 MB). Esto limita la cantidad de datos cifrados con la misma clave.
    - **reneg-pkts n**: renegocia tras n paquetes (por defecto deshabilitado). Es otra forma de fijar el límite de tráfico antes de cambiar clave.
    - **reneg-sec max [min]**: renegocia tras transcurrir “max” segundos (y al menos “min” segundos). Por defecto reneg-sec 3600 (1 hora), lo que suele significar rekey cada hora. El valor usado es aleatorio entre el 90% y 100% de max. Si se fijan valores distintos en cliente/servidor, prevalece el más bajo.

```
client
dev tun
proto udp
remote 192.168.1.120 1194
nobind
persist-key
persist-tun
ca /etc/openvpn/keys/ca.crt
cert /etc/openvpn/keys/micliente1.crt
key /etc/openvpn/keys/micliente1.key
remote-cert-tls server
tls-auth /etc/openvpn/keys/ta.key 1
cipher AES-256-GCM
auth SHA512
tls-version-min 1.2
```

```
tls-cipher TLS-DHE-RSA-WITH-AES-256-GCM-SHA384
# Renegociación de claves
reneg-sec 3600
reneg-bytes 500000000
reneg-pkts 1000000
verb 9   # 0-9, 9 más verboso
```

Figura 33. Modificación de la configuración del cliente

Si bien se podría forzar el uso de la versión TLS 1.3, la realidad es que en general el cliente cuenta con un gran número de parámetros que hacen que la conexión sea completamente segura. No se ha comentado acerca de los parámetros de configuración para la fijación de protocolos SSL/TLS ni de Ciphersuite específicas puesto que ya se hacen uso de estas en el propio ejemplo.

2.2.2. Windows

Al igual que con un cliente Linux, también es posible conectar clientes OpenVPN en Windows, descargando el cliente adecuado desde la página oficial. Primero, se copia la plantilla de configuración cliente.ovpn a la carpeta config y se renombra como micliente2.ovpn. A continuación, se generan en el servidor los certificados del nuevo cliente siguiendo los pasos anteriores y se copian de forma segura los archivos micliente2.crt, micliente2.key, ta.key y ca.crt al sistema cliente. Por último, se edita el archivo de configuración para definir parámetros como el servidor remoto, protocolos, cifrado y autenticación, dejando el cliente listo para conectarse al servidor OpenVPN.

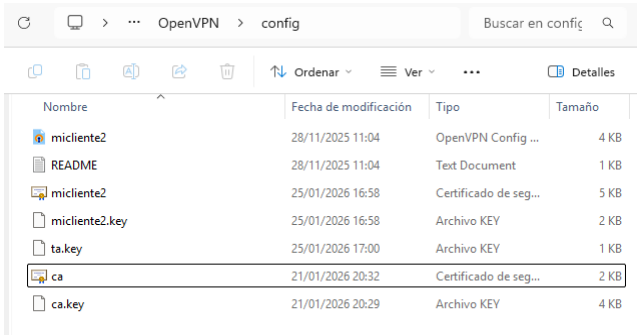


Figura 34. Instalación de certificados y autoridades de certificación

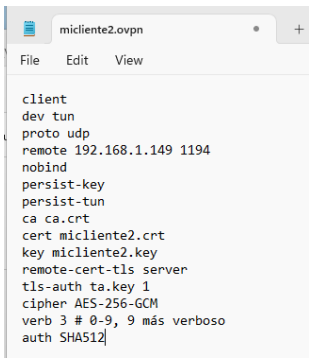


Figura 35. Contenido del fichero 'micliente2.ovpn'

Desde el cliente OpenVPN GUI, se conecta al servidor haciendo clic derecho sobre el icono y seleccionando Conectar, mostrando un log de la conexión con opciones de desconectar y reconectar. Una vez establecida, se pueden probar los servicios corporativos accediendo a ellos mediante la IP privada del servidor.

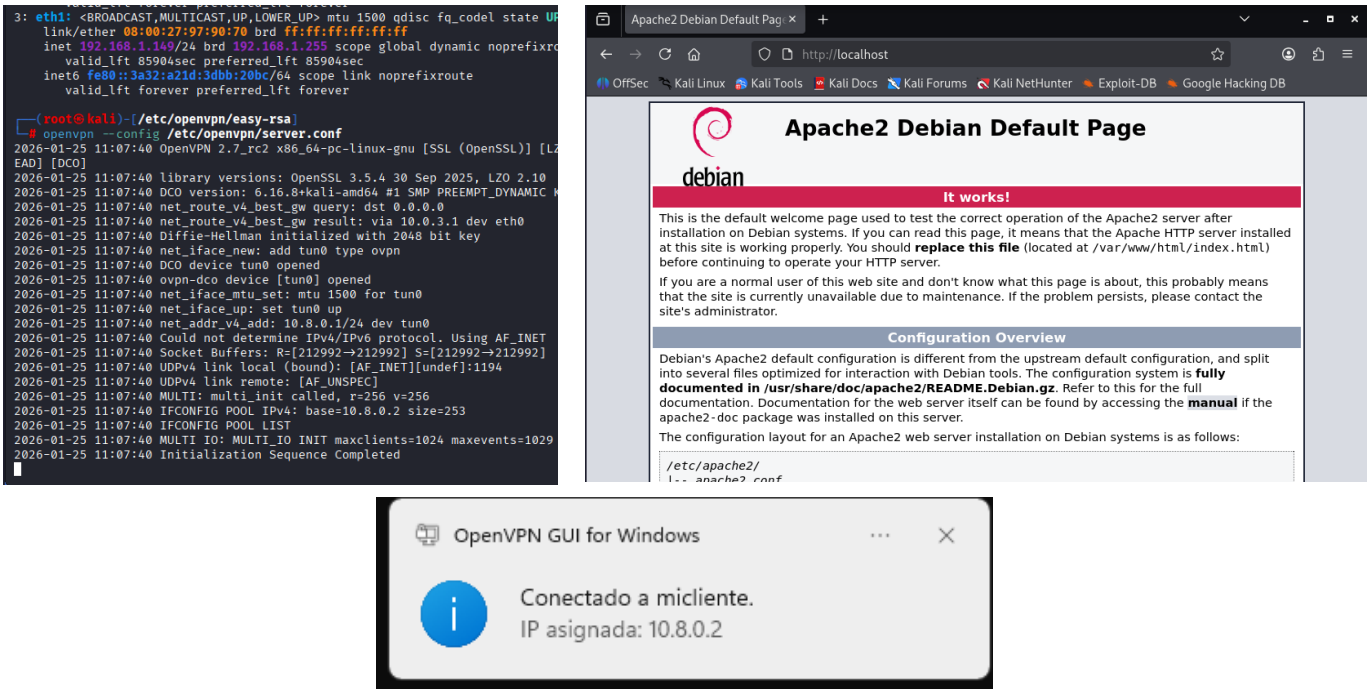


Figura 36. Conexión establecida con el servidor VPN y acceso a los servicios ofrecidos

En este punto, debemos considerar que, aunque los clientes de la VPN estén en el mismo rango de subred, por defecto no siempre pueden comunicarse entre sí, ya que OpenVPN suele deshabilitar el enrutamiento de cliente a cliente por seguridad.

Para habilitar la comunicación entre clientes, es necesario configurar el parámetro client-to-client en el archivo de configuración del servidor, lo que permite que los paquetes enviados por un cliente puedan ser recibidos directamente por otro. Además, se deben asegurar rutas correctas y que no existan reglas de firewall que bloqueen el tráfico interno de la VPN. En los clientes, basta con mantener la configuración estándar que apunta al servidor y aceptar la ruta de la VPN.

```
Adaptador desconocido OpenVPN Connect DCO Adapter:

   Sufijo DNS específico para la conexión. . :
   Vínculo: dirección IPv6 local. . . : fe80::2225:f51b:322c:c409%18
   Dirección IPv4. . . . . . . . . . . . . . : 10.8.0.2
   Máscara de subred . . . . . . . . . . . . : 255.255.255.0
   Puerta de enlace predeterminada . . . . . :
```

```
5: tun1: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/none
    inet 10.8.0.3/24 brd 10.8.0.255 scope global tun1
       valid_lft forever preferred_lft forever
    inet6 fe80::a28e:d83f:a622:ea90/64 scope link stable-privacy proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figura 37. Rango de direcciones IP de los clientes

```
…
ifconfig-pool-persist /var/log/openvpn/ipp.txt

# --- PARA LA CONEXIÓN CON OTROS CLIENTES ---
client-to-client

cipher AES-256-GCM
…
```

Figura 38. Modificación de la configuración del servidor para permitir conexiones entre clientes

Para probar la conectividad, se accedió al servidor Apache de un cliente a través de otro, utilizando el túnel VPN. Estas pruebas confirmaron que al habilitar client-to-client se permite la comunicación interna segura entre clientes dentro de la VPN.

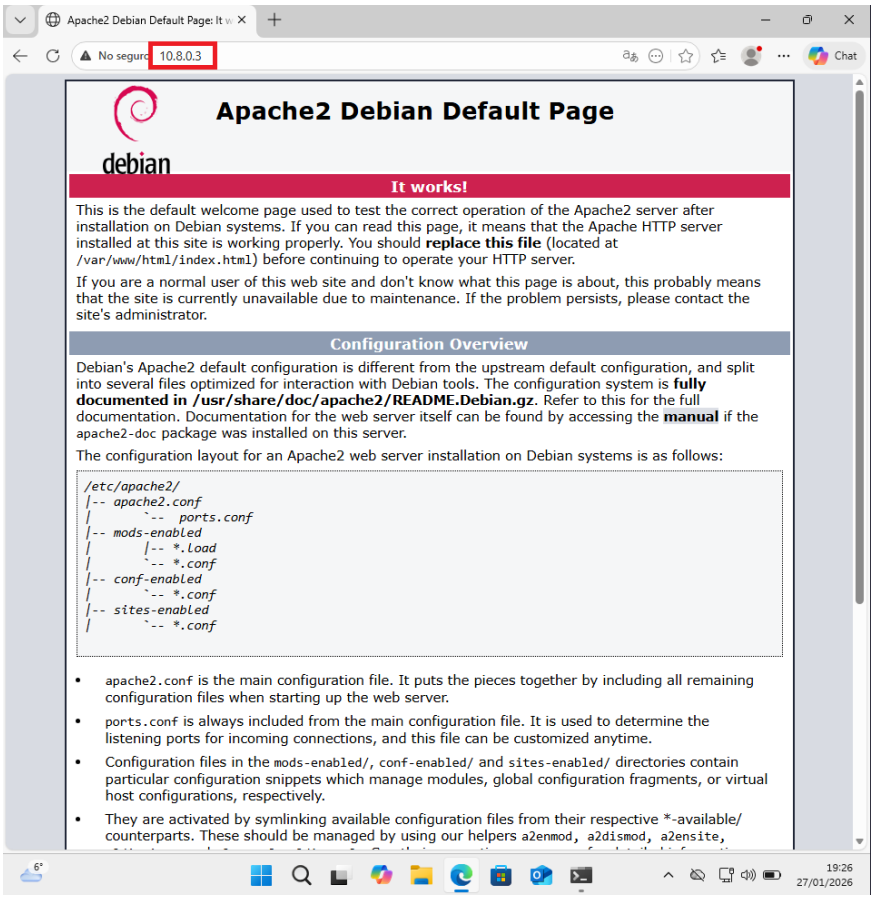


Figura 39. Acceso al servicio alojado en el cliente VPN 10.8.0.3 desde el cliente 10.8.0.2

2.2.3. Android

Se instalaron los certificados y la autoridad de certificación copiando de forma segura client.crt, client.key y ca.crt en una carpeta del dispositivo Android. A continuación, se modificó el fichero client.conf para que apuntara correctamente a la ubicación de estos archivos en el dispositivo y se verificó que la dirección IP del servidor VPN fuera correcta. Posteriormente, se renombró client.conf como client.ovpn y se colocó en la misma carpeta de los certificados. Esta modificación es fundamental para que el cliente Android pueda establecer correctamente la conexión segura con el servidor VPN.

Figura 40. Fichero de configuración del cliente modificado

Si es necesario aumentar la compatibilidad, en el fichero de configuración del servidor server.conf se deben deshabilitar las líneas correspondientes a tls-auth, AES-256-GCM, SHA512, tls-version-min 1.2 y tls-cipher TLS-DHE-RSA-WITH-AES-256-GCM-SHA384, y, a continuación, añadir la línea cipher AES-128-CBC. Esta modificación permite que el servidor utilice un cifrado más compatible con clientes que no soportan las suites de seguridad más recientes, asegurando así la conexión con un mayor número de dispositivos.

Para finalizar, se importó el archivo .ovpn en OpenVPN Connect y se estableció la conexión de manera exitosa. Para verificar su correcto funcionamiento, además se accedió al servidor Apache alojado en el servidor a través del túnel VPN.

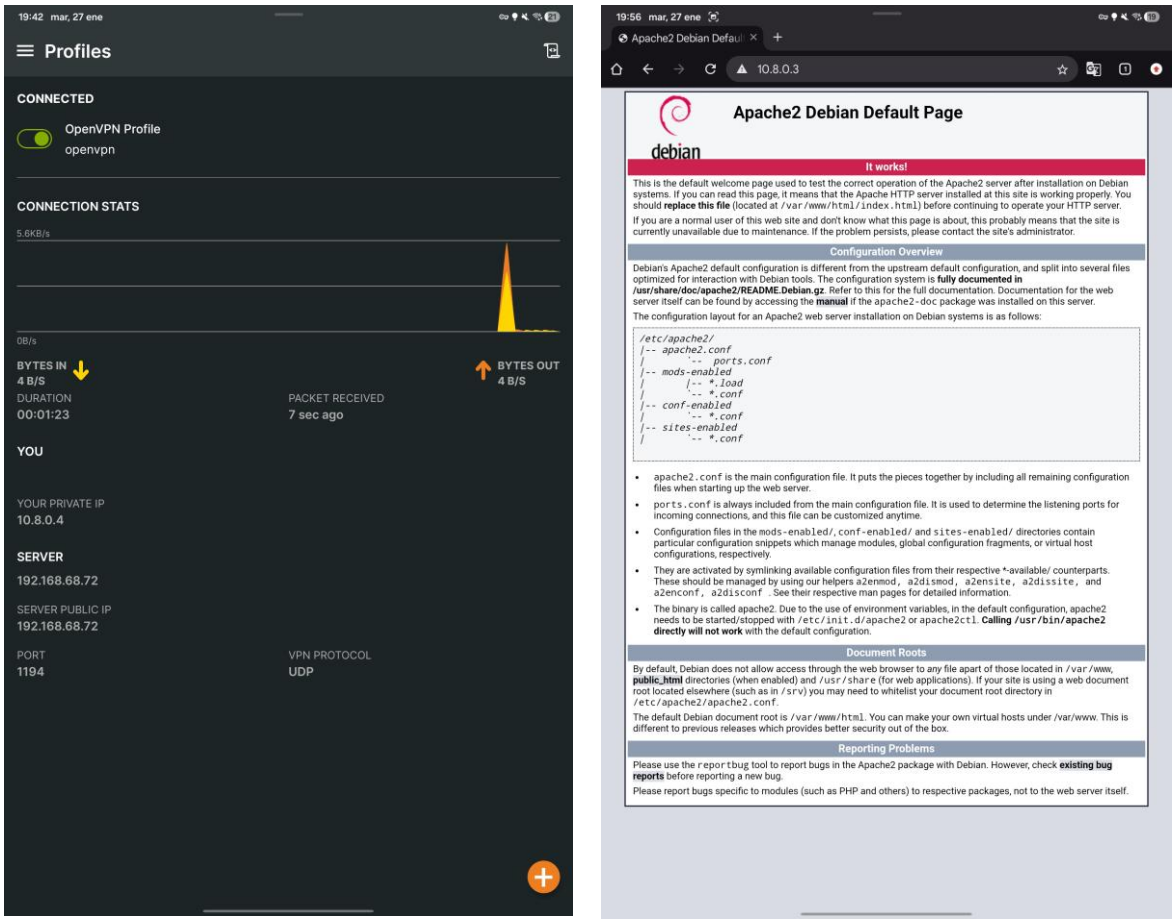


Figura 41. Conexión con el servidor VPN y acceso a los servicios corporativos

2.3. OpenVPN Access Server

Este apartado no se ha realizado; tal y como pone en la memoria, en su sustitución, se ha optado por realizar la tarea extra 'VPN personales'.

2.4. VPN personales (Tarea Extra)

Para este apartado, es necesario registrarse en el servicio No-IP, el cual nos ofrece un dominio gratuito que se vincula a una dirección IP dinámica mediante DDNS. En nuestro caso, la IP a utilizar será la del router.

Se ha decidido llamar al dominio 'vpnjuarobmenciber'. Será de tipo A porque este tipo de registro asocia directamente un nombre de dominio con una dirección IP específica, lo que permite acceder al router de manera directa. Se ha escogido que pertenezca a ddns.net porque es un subdominio proporcionado por No-IP que facilita la actualización automática

de la IP dinámica. Además, es fundamental activar el Dynamic DNS de nuestro nuevo dominio para que los cambios de IP se reflejen automáticamente y el acceso remoto funcione de manera continua.

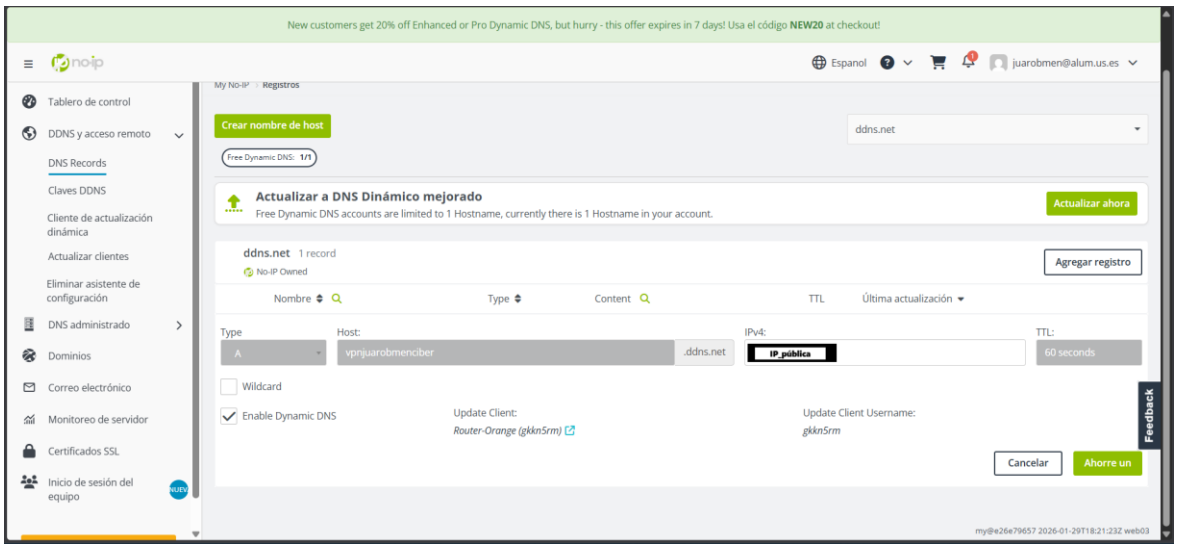


Figura 42. Configuración del dominio DynDNS en No-IP

A continuación, seleccionamos la plataforma donde se ejecutará el servicio No-IP. Dado que nuestro router no cuenta con un sistema operativo instalado, elegimos la opción 'DDNS Compatible Device'. Seguidamente, asignamos un nombre descriptivo que nos permita identificar fácilmente el equipo. Finalmente, se nos proporcionan las nuevas credenciales de DDNS, compuestas por usuario y contraseña, que serán necesarias para la configuración del servicio.

Figura 43. Plataforma en la que se ejecutará el servicio No-IP

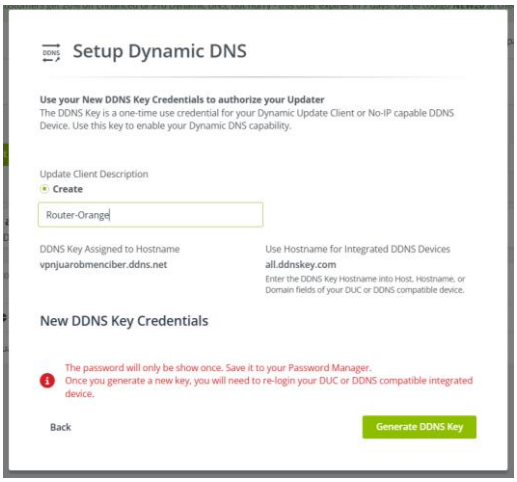


Figura 44. Nombre descriptivo del equipo

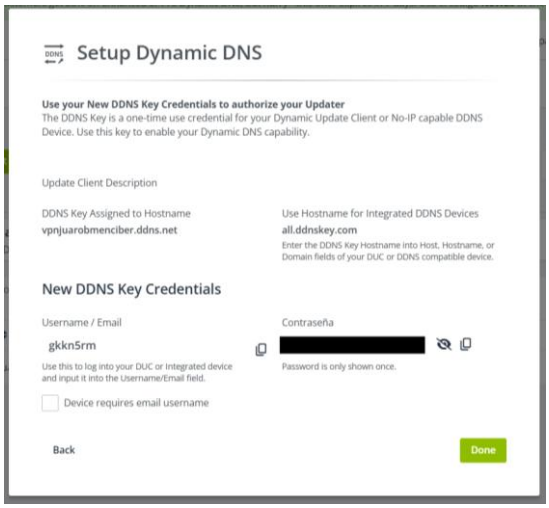


Figura 45. Credenciales de la clave DDNS

Accediendo al Livebox como administrador, debemos dirigirnos a la sección de DynDNS dentro de la configuración avanzada. Allí añadiremos una nueva entrada, especificando el servicio DDNS utilizado (en este caso, No-IP), las credenciales de acceso (usuario y contraseña) y el nombre de host completo asociado.

Figura 46. Configuración del DynDNS en el Livebox

Para finalizar, es necesario abrir el puerto correspondiente al servicio OpenVPN en el cortafuegos, permitiendo así la conectividad remota desde Internet hacia nuestro servidor VPN. Para ello, accedemos a la sección NAT/PAT y añadimos una nueva entrada, indicando el nombre del servicio (OpenVPN) y los puertos externos e internos, que en nuestro caso son ambos 1194. Además, debemos seleccionar UDP como protocolo de transporte y especificar la IP privada del servidor VPN dentro de nuestra LAN (192.168.1.148).

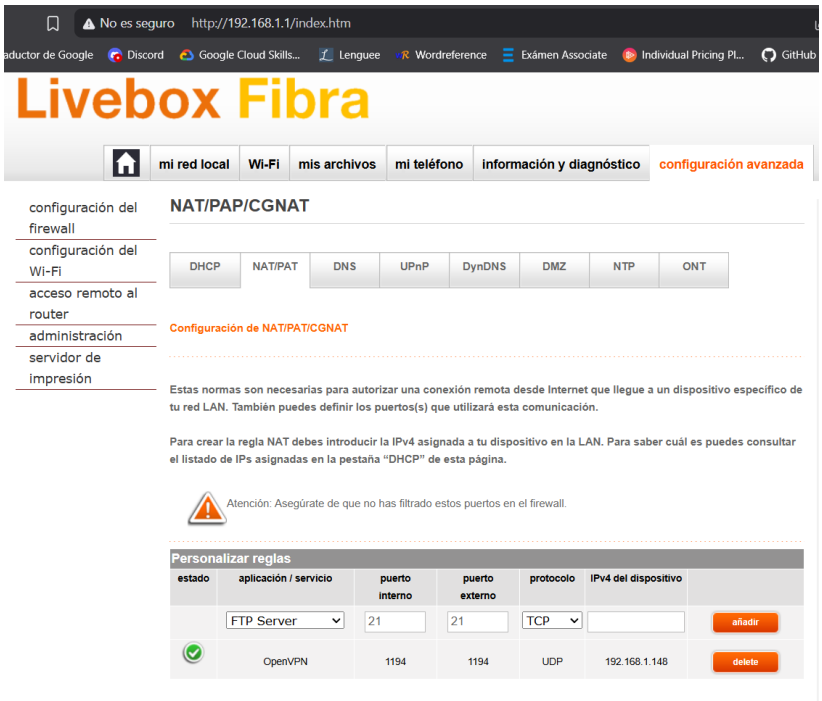


Figura 47. Configuración para las conexiones remotas en el Livebox

En nuestro caso, para simular una conexión al servidor desde fuera de la LAN, se ha utilizado la aplicación OpenVPN Connect en un dispositivo iOS (empleando lógicamente los datos móviles). En ella, al igual que se hizo previamente en el apartado de Android, se ha configurado un cliente VPN. Para ello, se ha importado un archivo 'conexion.ovpn', que además de la configuración del cliente, incluye los demás archivos necesarios mediante referencias '< >…</ >', como ca.crt, cliente1.crt, cliente1.key, ta.key, entre otros.

```
client
dev tun
proto udp
remote vpnjuarobmenciber.ddns.net 1194
resolv-retry infinite
nobind
persist-key
persist-tun
remote-cert-tls server
cipher AES-256-GCM
auth SHA512
tls-version-min 1.2
<ca>
-----BEGIN CERTIFICATE-----
MIIE2zCCA8OgAwIBAgIUfcJPuAaq5bv8fr0cLtMf/S37UQUwDQYJKoZIhvcNAQEL
...
oI+rcn1eH3CzCshB5h0rCAcs4B5wnh88lssM7/ZIJzSdPWX+qoSdLJtRqyswIdI=
-----END CERTIFICATE-----
</ca>
```

```
<cert>
-----BEGIN CERTIFICATE-----
MIIE7TCCA9WgAwIBAgIQGfvaKdAY8HpaLbN5AXpfYzANBgkqhkiG9w0BAQsFADCB
...
NHrAvA0AI7rnJEJIaDEu4t4=
-----END CERTIFICATE-----
</cert>
<key>
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQCIREeTtxwYpcna
...
ZtXIy/aABeArBWRMPhvoHIM=
-----END PRIVATE KEY-----
</key>
<tls-auth>
-----BEGIN OpenVPN Static key V1-----
c927fda7baab4bec849335dd525a8358
...
334ad0ead5016ba3268389d5549f9096
-----END OpenVPN Static key V1-----
</tls-auth>
key-direction 1
```

Figura 48. Contenido del fichero conexion.ovpn

Tras el arranque, es posible verificar que la conexión se ha establecido correctamente y, a su vez, acceder a los servicios corporativos alojados en el servidor, como por ejemplo el servidor Apache. Además, en los registros de salida del servidor VPN se pueden visualizar todas las configuraciones y parámetros de la conexión establecida con el dispositivo móvil.

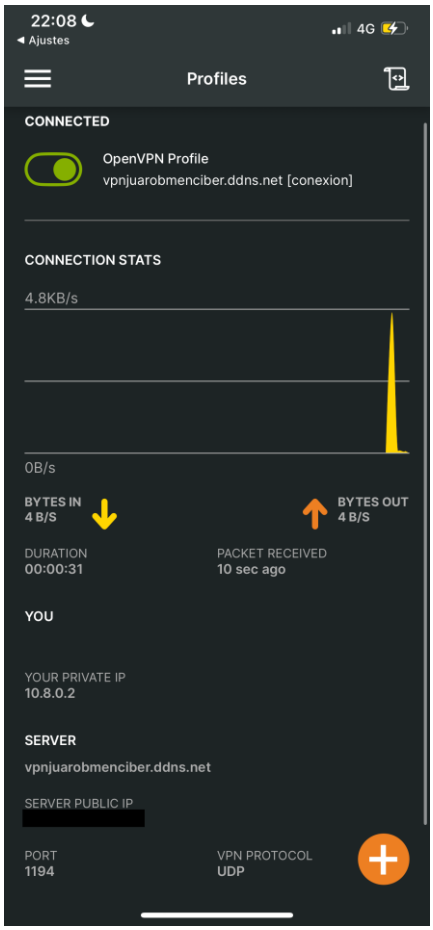
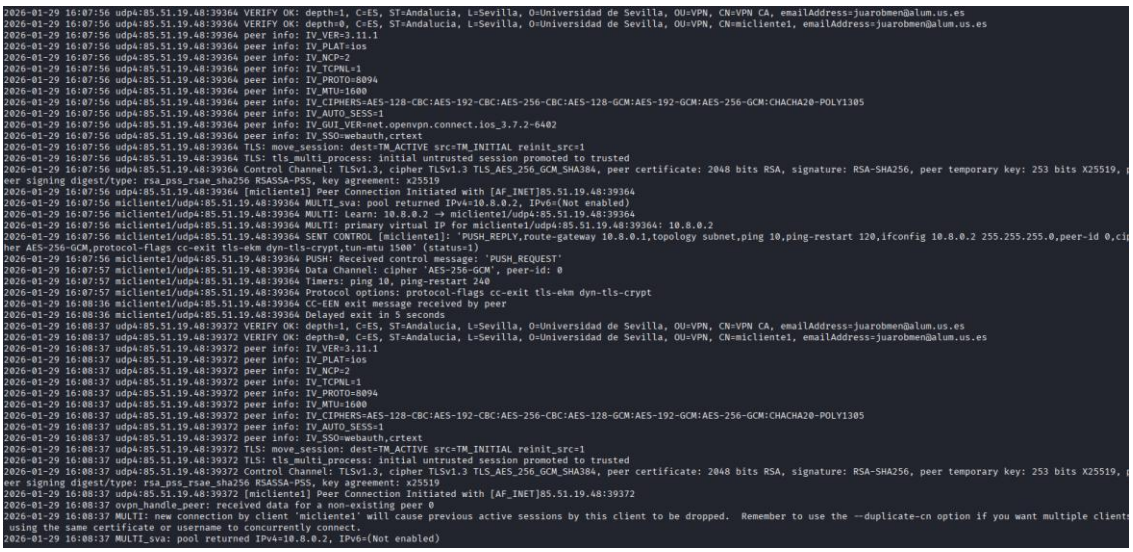


Figura 49. Importación del .ovpn y establecimiento de la conexión con el servidor

Figura 50. Acceso a los servicios desplegados en la red corporativa fuera de la red local (desde el cliente)



```
2026-01-29 16:07:56 udp4:85.51.19.48:39364 VERIFY OK: depth=1, C=ES, ST=Andalucia, L=Sevilla, O=Universidad de Sevilla, OU=VPN, CN=VPN CA, emailAddress=juarobmen@alum.us.es
2026-01-29 16:07:56 udp4:85.51.19.48:39364 VERIFY OK: depth=0, C=ES, ST=Andalucia, L=Sevilla, O=Universidad de Sevilla, OU=VPN, CN=micliente1, emailAddress=juarobmen@alum.us.es
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_VER=3.11.1
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_PLAT=ios
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_NCP=2
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_TCPNL=1
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_PROTO=8094
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_MTU=1600
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_CIPHERS=AES-128-CBC:AES-192-CBC:AES-256-CBC:AES-128-GCM:AES-192-GCM:AES-256-GCM:CHACHA20-POLY1305
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_AUTO_SESS=1
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_GUI_VER=net.openvpn.connect.ios_3.7.2-6402
2026-01-29 16:07:56 udp4:85.51.19.48:39364 peer info: IV_SSO=webauth,crtext
2026-01-29 16:07:56 udp4:85.51.19.48:39364 TLS: move_session: dest=TM_ACTIVE src=TM_INITIAL reinit_src=1
2026-01-29 16:07:56 udp4:85.51.19.48:39364 TLS: tls_multi_process: initial untrusted session promoted to trusted
2026-01-29 16:07:56 udp4:85.51.19.48:39364 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, peer certificate: 2048 bits RSA, signature: RSA-SHA256, peer temporary key: 253 bits X25519,
eer signing digest/type: rsa_pss_rsae_sha256 RSASSA-PSS, key agreement: x25519
2026-01-29 16:07:56 udp4:85.51.19.48:39364 [micliente1] Peer Connection Initiated with [AF_INET]85.51.19.48:39364
2026-01-29 16:07:56 micliente1/udp4:85.51.19.48:39364 MULTI_sva: pool returned IPv4=10.8.0.2, IPv6=(Not enabled)
2026-01-29 16:07:56 micliente1/udp4:85.51.19.48:39364 MULTI: Learn: 10.8.0.2 -> micliente1/udp4:85.51.19.48:39364
2026-01-29 16:07:56 micliente1/udp4:85.51.19.48:39364 MULTI: primary virtual IP for micliente1/udp4:85.51.19.48:39364: 10.8.0.2
2026-01-29 16:07:56 micliente1/udp4:85.51.19.48:39364 SENT CONTROL [micliente1]: 'PUSH_REPLY,route-gateway 10.8.0.1,topology subnet,ping 10,ping-restart 120,ifconfig 10.8.0.2 255.255.255.0,peer-id 0,cip
her AES-256-GCM,protocol-flags cc-exit tls-ekm dyn-tls-crypt,tun-mtu 1500' (status=1)
2026-01-29 16:07:56 micliente1/udp4:85.51.19.48:39364 PUSH: Received control message: 'PUSH_REQUEST'
2026-01-29 16:07:57 micliente1/udp4:85.51.19.48:39364 Data Channel: cipher 'AES-256-GCM', peer-id: 0
2026-01-29 16:07:57 micliente1/udp4:85.51.19.48:39364 Timers: ping 10, ping-restart 240
2026-01-29 16:07:57 micliente1/udp4:85.51.19.48:39364 Protocol options: protocol-flags cc-exit tls-ekm dyn-tls-crypt
2026-01-29 16:08:36 micliente1/udp4:85.51.19.48:39364 CC-EEN exit message received by peer
2026-01-29 16:08:36 micliente1/udp4:85.51.19.48:39364 Delayed exit in 5 seconds
2026-01-29 16:08:37 udp4:85.51.19.48:39372 VERIFY OK: depth=1, C=ES, ST=Andalucia, L=Sevilla, O=Universidad de Sevilla, OU=VPN, CN=VPN CA, emailAddress=juarobmen@alum.us.es
2026-01-29 16:08:37 udp4:85.51.19.48:39372 VERIFY OK: depth=0, C=ES, ST=Andalucia, L=Sevilla, O=Universidad de Sevilla, OU=VPN, CN=micliente1, emailAddress=juarobmen@alum.us.es
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_VER=3.11.1
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_PLAT=ios
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_NCP=2
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_TCPNL=1
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_PROTO=8094
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_MTU=1600
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_CIPHERS=AES-128-CBC:AES-192-CBC:AES-256-CBC:AES-128-GCM:AES-192-GCM:AES-256-GCM:CHACHA20-POLY1305
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_AUTO_SESS=1
2026-01-29 16:08:37 udp4:85.51.19.48:39372 peer info: IV_SSO=webauth,crtext
2026-01-29 16:08:37 udp4:85.51.19.48:39372 TLS: move_session: dest=TM_ACTIVE src=TM_INITIAL reinit_src=1
2026-01-29 16:08:37 udp4:85.51.19.48:39372 TLS: tls_multi_process: initial untrusted session promoted to trusted
2026-01-29 16:08:37 udp4:85.51.19.48:39372 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, peer certificate: 2048 bits RSA, signature: RSA-SHA256, peer temporary key: 253 bits X25519,
eer signing digest/type: rsa_pss_rsae_sha256 RSASSA-PSS, key agreement: x25519
2026-01-29 16:08:37 udp4:85.51.19.48:39372 [micliente1] Peer Connection Initiated with [AF_INET]85.51.19.48:39372
2026-01-29 16:08:37 ovpn_handle_peer: received data for a non-existing peer 0
2026-01-29 16:08:37 MULTI: new connection by client 'micliente1' will cause previous active sessions by this client to be dropped.  Remember to use the --duplicate-cn option if you want multiple client
 using the same certificate or username to concurrently connect.
2026-01-29 16:08:37 MULTI_sva: pool returned IPv4=10.8.0.2, IPv6=(Not enabled)
```

Figura 51. Salida del servidor openVPN tras la conexión del cliente